

claude-rv32 驗證報告

RV32IMC_Zicsr 管線化 RISC-V 軟核之完整驗證活動彙整

受測設計 (DUT)：rv32_core / rv32_soc —— 6 級暫存器管線、branch-in-EX、BTB 動態預測、全前遞、I\$/D\$ 經 AXI4。

文件版本：2026-06-06 | 目標平台：PYNQ-Z2 (Zynq-7000 XC7Z020-1CLG400C)

arch-test 全通過

Spike lockstep 全乾淨

CoreMark CRC 驗證

覆蓋率 82.8%

形式化 78/78

找到並修復 9 個錯誤

目錄

- 摘要
- 驗證策略
- 測試環境
- 功能測試結果
- 效能基準 (亦為正確性測試)
- 找到並修復的錯誤
- 程式碼覆蓋率
- 形式化驗證
- 已知限制與未來工作
- 如何重現
- 結論

1. 摘要

驗證活動	工具	結果
架構相容性 (riscv-arch-test)	iverilog + VCS-MX	ui 39/39、um 8/8、uc 1/1、mi 14/14*
Spike 逐指令 lockstep 共模擬	<code>scripts/lockstep.py</code>	全套 lockstep 乾淨
定向 / 中斷 / 壓力 測試	iverilog / VCS	全通過 (自檢 + lockstep)
約束隨機	riscv-dv (pyflow)	每個 seed lockstep 乾淨
隨機 load/store 壓力	影子記憶體比對	通過 (多 seed)
效能基準 — Dhrystone 2.1	on-core 週期計數	≈0.71 DMIPS/MHz
效能基準 — CoreMark 1.0	自帶 CRC 驗證	2.61 (-O2) / 2.75 (-O3) CoreMark/MHz
程式碼覆蓋率	VCS <code>-cm</code> (line/cond/toggle/fsm/branch)	82.8% 總分
形式化驗證	riscv-formal + OSS CAD Suite	78/78 項通過 (RV32IMC+CSR)

* 核心通過全部 14 項 `rv32mi`；`breakpoint` 僅排除於 **Spike** 交叉比對 (Spike 對選用的 debug-trigger 規範自身判定不一致，核心行為本身正確)。一行指令即可跑完全部：`scripts/run_regression.sh`。

結論先講：功能正確性在四個獨立層級上建立——定向測試、架構相容套件、對黃金模型的逐指令等價，以及有界模型形式化證明——並由兩個自我驗證的基準測試佐證。整個過程**找到並修復 9 個錯誤** (第 6 節)，其中最關鍵的幾個是由 **CoreMark** 與**形式化**揪出，而非定向套件。

2. 驗證策略

核心遵循「**每階段只引入一項新複雜度**」的專案原則逐步帶起，驗證採分層設計，使每一層抓不同類別的缺陷：

- 1. 定向測試——針對單一功能、答案已知的手寫程式；最易撰寫與除錯，作為每階段的第一道關。
- 2. 架構相容套件 (riscv-arch-test) ——標準 `rv32u[imc]` 與 `rv32mi` ；對社群參考確認 ISA 一致性。
- 3. Spike lockstep——每條退役指令的 PC 與暫存器寫入，逐指令與 Spike 黃金模型比對；能精準定位發散的那一條指令，抓到大多數管線錯誤。
- 4. 隨機與約束隨機——`memstress` （隨機 load/store 對照影子記憶體）與 riscv-dv （約束隨機指令流），到達定向測試碰不到的狀態。
- 5. 效能基準——Dhrystone 與 CoreMark ；CoreMark 的 CRC 自檢使其同時是正確性測試，並確實揪出一個潛在 D-cache 錯誤。
- 6. 程式碼覆蓋率——以 VCS 結構覆蓋率找未測 RTL，再以定向測試補齊可達成的缺口，並誠實分析結構性殘餘。
- 7. 形式化——riscv-formal 在 RVFI 退役介面上做有界模型證明；在邊界內對所有輸入成立，與任何刺激無關。

第 2～7 層都建立在同一個可觀測量上：退役指令串流。

3. 測試環境

元件	細節
模擬器	Icarus Verilog <code>-g2012</code> （快速預設）與 Synopsys VCS-MX O-2018.09-SP2 （ <code>SIM=vcs</code> ）；兩者跑同一個 <code>tb_top.sv</code> ，全套結果一致。
黃金模型	Spike (riscv-isa-sim) ， <code>--log-commits</code> ，ISA <code>rv32imc_zicsr_zifencei</code> 。
lockstep 比對器	<code>scripts/lockstep.py</code> ——將核心的 <code>RETIRE pc=... xN=...</code> trace （ <code>+define+RV_TRACE</code> ）與 Spike commit log 比對。
工具鏈	xpack <code>riscv-none-elf-gcc</code> 14.2.0 （ <code>-march=rv32imc -mabi=ilp32 -nostdlib</code> ）。
覆蓋率	VCS <code>-cm line+cond+toggle+fsm+branch</code> ，urg 合併。
形式化	YosysHQ OSS CAD Suite (yosys + SymbiYosys + boolector/yices/z3/btormc) + riscv-formal 框架。
合成（背景）	AMD Vivado 2024.2 ，XC7Z020-1CLG400C 。

VCS-MX 主機修補：VCS-2018 參考 `pthread_yield()` （glibc ≥ 2.34 已移除），故以一行 `sched_yield()` 包裝（`sim/vcs_shim/` ）透過 `-LDFLAGS` 傳入，否則 `simv` 在 Ubuntu 22.04 無法連結。

4. 功能測試結果

4.1 架構相容套件 (riscv-arch-test)

使用標準 `-p-`（機器模式）環境；各測試以 `tohost` 自檢。

套件	內容	核心	Spike lockstep
<code>rv32ui-p-*</code>	RV32I 基礎整數	39 / 39	乾淨
<code>rv32um-p-*</code>	M 擴充 (mul/div)	8 / 8	乾淨
<code>rv32uc-p-*</code>	C 擴充 (壓縮指令)	1 / 1	乾淨 (186 指令)
<code>rv32mi-p-*</code>	機器模式：CSR / trap / 未對齊 / 非法	14 / 14	13/14 (breakpoint 為 Spike-only)

Phase 5 期間以兩種快取關聯度（直接對映與 2 路）以及多種注入的記憶體等待週期（`iwait` / `dwait` 0/1/3/5/7）重新驗證。執行器：`scripts/run_rv32{ui,um,uc,mi}.sh`。

4.2 Spike 逐指令 lockstep 共模擬

Spike 為黃金參考；逐指令退役比對與週期數無關（比的是退役 PC + 暫存器寫入的序列）。任何取指或執行錯誤都會在**確切發散的那條指令**被標出。此機制定位了暫存器飄移、快取讀寫競態、store→load 前遞，以及壓縮跳躍等錯誤（第 6 節）。

一個值得記錄的工具細節：比對器原本以 `rv32i_zifencei` 啟動 Spike，無法解碼壓縮的 `rvc` 測試——Spike 在 PC 0 陷入 trap 迴圈、產生數 GB 的 log（被誤報為發散）。預設 ISA 已修正為 `rv32imc_zifencei`。

4.3 定向、中斷與壓力測試

全部自檢；結構允許者即 lockstep 乾淨（結果依賴即時計數器／中斷時序者無法與 Spike 逐位元比對，改採自檢）。

測試	檔案 / 執行器	涵蓋內容
smoke、 pltest	<code>smoke.S</code> 、 <code>pltest.S</code>	帶起：算術、分支、各種 load/store 寬度、前遞、load-use 停頓。
中斷	<code>irqtest.S</code>	timer + 外部中斷；handler 檢查 <code>mcause</code> 、清除各來源（自檢）。
快取衝突	<code>cachestress.S</code>	強迫 16 條 line 進入同一 set（驅逐/回填/LRU churn）；DM 與 2 路皆通過。
D-cache 飄移	<code>dcache_drift.c</code>	精確的 <code>lw A; lw B</code> 同 tag 樣式——在有錯核心上失敗、修復後通過且 lockstep 乾淨（具證明力的回歸）。
隨機 LSU	<code>memstress.c</code>	隨機 byte/half/word load/store 對照影子記憶體；可參數化 seed / 迭代數 / 關聯度 / 寫入 FIFO 深度。
串流	<code>memstream.S</code>	冷 miss 串流（驗證 burst 回填）。
CSR / trap	<code>csrtrap.c</code> + <code>csrtrap_vec.S</code>	CSR 寫入掃描、全部 6 種 CSR 指令型式、每一種同步 trap 原因、M→U→M 特權往返——自檢 + Spike lockstep。
CLINT 計時器	<code>clinttest.c</code>	mtime/mtimecmp 往返、64 位元遞增進位、 <code>mtime≥mtimecmp</code> 比較兩個方向（經 <code>mip.MTIP</code> ，自檢）。
BTB 壓力	<code>btbstress.c</code>	512 條 taken 分支掃過全部 128 個 BTB set + 計算式跳躍表；lockstep 乾淨（BTB 不影響結果），兼作損毀回歸。

4.4 約束隨機 — riscv-dv

以 YosysHQ/Google 的 **riscv-dv** 產生器（pyflow）產生約束隨機指令流；每個 seed 在核心上執行並對 Spike lockstep（`run_riscv_dv.sh`、`tests/riscv-dv/`）。範圍為 ALU/分支指令流（產生器的 LSU/資料頁串流需要帶起階段尚未提供的完整記憶體模型）。逐 seed 產生有時間上限，失敗即略過，故每次執行會回報產生並比對了幾個 seed。

5. 效能基準（亦為正確性測試）

兩個基準都在核心上執行，並以 Zicsr 的 `mcycle` / `minstret` 計數器自我計時。

基準	建置	結果	備註
Dhrystone 2.1	- 02	≈0.71 DMIPS/MHz	時序收斂前為 0.724；暫存化 redirect + 管線化乘法器以約 2% 換取 Fmax。
CoreMark 1.0	- 02	2.61 CoreMark/MHz	自我驗證（對結果清單算 CRC）。
CoreMark 1.0	- 03	2.75 CoreMark/MHz	更多展開/內聯。

CoreMark 的 CRC 是對其自身結果清單計算、且須與期望值相符，故正確的分數**同時是正確性證明**。此結果落在已發表的 Ibex 區間（2.4–3.1 CoreMark/MHz）內——即與成熟量產核心同一效能級別（見 [compare_ibex.html](#)）。**CoreMark 在套件中的價值，正是揪出了一個 D-cache 偽命中錯誤**（第 6 節）——這個錯誤連所有 arch-test 與 Dhrystone 都漏掉了。

6. 找到並修復的錯誤

整個過程的價值，最能由「它抓到了什麼」來呈現。每個錯誤都標註是哪一層揪出的。

#	錯誤	由何層揪出	修法
1	停頓時 IF2 指令遺失 (live <code>i_imem_rdata</code> 已前進)	retire trace / lockstep (P1)	IF2 一格 skid buffer
2	未對齊載入偵測漏掉 <code>LHU</code> (funct3=101)	<code>rv32mi</code> <code>ma_addr</code> (P2)	補上該 funct3 分支
3	RVC 誤解碼： <code>C.ADDI4SPN</code> / <code>C.LW</code> 的 rd' 欄位、 <code>C.J</code> / <code>C.JAL</code> 立即值 (offset 0 → 無窮自迴圈)、壓縮非法指令的 mtval、mret 至壓縮目標的 mepc 對齊	lockstep on <code>rvc</code> (P4)	修正 RVC 展開器 + 攜帶原始位元 / 保留 mepc bit 1
4	多週期停頓時暫存器堆飄移——同步讀的 <code>rf_rdata</code> 追到下一條指令的讀位址；任何因快取 miss 被held 在 EX 的指令會讀到錯的暫存器	lockstep (P5)	於 EX 第一週期擷取運算元 (<code>ex_fresh</code>) 並 held 住
5	D-cache：store-hit 合併讀到錯的 line；連續 store→load 遺失 load；同位址 store→load 讀寫競態	lockstep / trace (P5)	於 STORE/FILL 完成時接受；在 CMP 更新 line
6	I-cache 每條 miss 的 line 回填兩次 (同步陣列在回填後立刻仍是舊值)	FSDB + 週期分析器 (P5)	1 條 line 的 fill buffer，與陣列並排做組合比對 (週期 -37%)
7	D-cache 偽命中 (spurious tag-hit) ——陣列以 當前索引讀取，但命中卻和已鎖存的 tag 比較；當當前位址飄移到另一條同 tag 已快取 line 時，一次 miss 可能回傳錯的 line 資料	CoreMark CRC + lockstep	以暫存的 <code>read_is_a</code> (鎖存讀索引 == 請求索引) 對命中加閘
8	多週期乘除法的 RVFI 運算元回報 (完成週期的前遞運算元 ≠ 單元實際鎖存者) ——屬驗證模型缺陷，非核心缺陷	形式化 (<code>insn_mul</code>)	由乘法器/除法器輸出其鎖存的運算元 (<code>rvfi_a/b</code>) 作為唯一真值來源
9	除法器交握——除法器以 <code>clk</code> 自由運轉，故在 D-cache 停頓期間可能完成、回到 idle 並重啟，洩漏舊結果	形式化 (<code>insn_div</code>)	給除法器與乘法器相同的 <code>en(!mem_stall)</code> / <code>flush(redirect)</code> 紀律

觀察

- **lockstep** 抓到最大一類（1-6）——產生錯誤退役值的管線/快取冒險，並在精確指令處標出。
- **CoreMark**（#7）到達了一種存取樣式（指標追逐、連續同 tag 載入），是 arch-test 與 Dhrystone 從未碰到的。
- **形式化**（#8-9）到達了沒有任何刺激會走到的狀態——除法撞上 D-cache 停頓——其結果是一個更穩健的除法器交握。

7. 程式碼覆蓋率

以 VCS 量測（`-cm line+cond+toggle+fsm+branch`，urg 合併整個回歸套件）；完整逐模組表見 `COVERAGE.md`。

指標	覆蓋率
總分	82.8%
Line	92.2%
Cond	87.0%
Branch	88.6%
Toggle	73.7%
FSM	72.2%

覆蓋率導向迴圈（量測 → 找最低模組 → 補一個定向測試 → 重新量測）補齊了兩個真正可達成缺口的模組：

- **csr.v 67 → 77%**——由 `csrtrap` 補齊（toggle 35 → 62%）。
- **clint.v 69 → 87%**——由 `clinttest` 補齊（toggle 67 → 92%）。

其餘偏低數字逐一被追溯為**結構性**因素，而非缺測試——這本身即是誠實而有用的結論：

- **btb toggle（約 51%）**——未翻轉的位元是恆定的高位位址（`lu_pc/up_pc/lu_target[31:14]`、`tag[23:6]`）：整個程式位於 `0x8000_0000` 的單一 256 KB 區域，故位元 [31:18] 永不改變。`btbstress` 填滿全部 128 個 entry，toggle 仍不動——證實瓶頸不在陣列。
- **icache/dcache FSM**——唯一未覆蓋的弧是 `REFILL→...` 落在 `if(!rst_n)` 那行，即回填中途的 reset，而僅在開機 reset 的測試平台無從觸發。
- **mem2axi cond**——半數是協定上不可能的列（`rlast` 而無 `rvalid`）；其餘需要恆為 ready 的單週期晶片內 BRAM 永遠不會產生的 slave 反壓。

可達成的覆蓋率已收斂；其餘受固定 SoC（單一 256 KB 區域、恆 ready 的 BRAM、僅開機 reset）所限。

8. 形式化驗證 (riscv-formal)

在 RVFI 退役介面上對 ISA 做有界模型檢查 (BMC)，使用 YosysHQ 的 riscv-formal 框架。核心的快取/AXI 位於 SoC，故形式化跑在核心乾淨的內部 `req` / `rvalid` 記憶體埠上；wrapper 給它一個由解算器自由選擇資料、單週期回應的「任意記憶體」。由於資料路徑會在 write-back 前壓掉 trap 指令，RVFI 紀錄改由 EX/MEM 與 MEM/WB 鎖存的影子產生，使 trap 指令也得以回報 (`rvfi_trap=1`)。完整細節見 `formal/README.md`。

`isa rv32imc` : 78/78 項通過。

檢查群組	數量	所證明者
<code>insn_*</code> (RV32I)	37	每條基礎整數指令的 rd / pc / 記憶體結果
<code>insn_c_*</code> (RV32C)	26	每條壓縮指令正確展開並執行
<code>insn_{mul,...,remu}</code> (RV32M)	8	M 解碼 / 運算元繞送 / 多週期停頓 / 前遞 / write-back (經 ALTOPS)
<code>csr_w_mscratch</code> 、 <code>csr_w_mtvec</code>	2	CSRRW/S/C 正確將 CSR 讀入 rd 並寫回 (可完全寫入的 M-CSR)
<code>csr_ill_fff</code>	1	存取不存在的 CSR 引發非法指令 trap
<code>reg</code>	1	<code>rvfi_rs1/rs2_rdata</code> 恆等於暫存器堆內容
<code>pc_fwd</code> 、 <code>pc_bwd</code>	2	連續指令間 PC 正確推進
<code>causal</code>	1	回報的暫存器讀取在因果上先於其使用
<code>unique</code>	1	每條指令恰好退役一次 (<code>rvfi_order</code> 單調)

M 擴充經由 ALTOPS。直接做乘法器等價證明 (DSP 33×33 對 spec 的 `*`) 是 SMT 經典難題、無法收斂。riscv-formal 的 `RISCV_FORMAL_ALTOPS` 模式將乘/除的結果原語——於 spec 與核心兩側——替換為每運算各異的廉價可逆函數，使 M 的解碼 / 運算元繞送 / 多週期停頓 / 前遞 / write-back 全部可證，而無需位元展開乘法器 (真實算術仍由 `rv32um` + CoreMark + lockstep 涵蓋)。

形式化本身揪出了上表的 #8 與 #9。整套以 `scripts/run_formal.sh` 執行 (自動 clone riscv-formal、產生檢查、並分發到多個解算器工作)。

已記錄的形式化未來範圍：trap CSR (`mepc` / `mcause` / `mtval` / `mstatus`) 與計數器 (`mcycle` / `minstret`) ——它們經由 trap 副作用 / WARL 欄位 / 每週期計數而變，需更多 RVFI 建模 (其行為目前由 `csrtrap` + lockstep 涵蓋) ——以及 AXI 匯流排/fault 檢查。

9. 已知限制與未來工作

- **形式化**：trap CSR、計數器與 AXI 匯流排檢查尚未納入（第 8 節）；M 的算術為抽象化（ALTOPS）而非逐位元證明（由模擬涵蓋）。
- **覆蓋率殘餘**為結構性（第 7 節）：再往上需組態變更（將程式碼散佈到多個位址區）或故障/停頓注入（執行中 reset、會停頓的 AXI slave 模型），而非更多定向測試。
- **中斷採自檢**（`irqtest`、`clinttest`）而非 lockstep，因 CLINT 的 `mtime` 速率與 Spike 不同；於形式化 wrapper 中綁定為無中斷。
- **無矽驗證、無安全/RAS 強化**——對此 FPGA 軟核屬範圍外（量產核心的領先處見 Ibex 比較）。

10. 如何重現

```
source scripts/env.sh

# 全部，一行 (arch-test + 定向 + 壓力 + 基準)，PASS/FAIL 表：
scripts/run_regression.sh          # iverilog (預設)；SIM=vcs 改用 VCS-MX
FULL=1 scripts/run_regression.sh   # + riscv-dv + 覆蓋率 (較慢)

# 個別層級：
scripts/run_rv32ui.sh              # 一個 arch-test 套件 (+ Spike lockstep)
scripts/run_csrtap.sh              # 一個定向測試 (自檢 + lockstep)
scripts/run_coremark.sh            # CoreMark (CRC 驗證)
scripts/run_coverage.sh            # VCS 覆蓋率 -> sim/cov_report/dashboard.html

# 形式化 (需 OSS CAD Suite；見 formal/README.md)：
export OSS_ENV=/path/to/oss-cad-suite/environment
scripts/run_formal.sh              # 78/78 項
```

11. 結論

就一顆 FPGA 軟核而言，claude-rv32 已達相當高的信心水準：**ISA 一致**（全部 `rv32u[imc]` / `rv32mi` arch-test）、與黃金模型**逐指令等價**（Spike lockstep 跨每個套件、定向測試與基準）、對完整 RV32IMC 指令集及代表性 CSR 語意**形式化證明**（78/78 項有界模型檢查），並由兩個自檢基準**效能驗證**於與成熟量產核心相同的級別。分層方法具體見效——找到並修復 9 個錯誤，包括一個只有 CoreMark 的存取樣式能觸及的潛在 D-cache 故障，以及一個只有形式化的窮舉狀態搜尋能曝露的除法器交握脆弱性。

